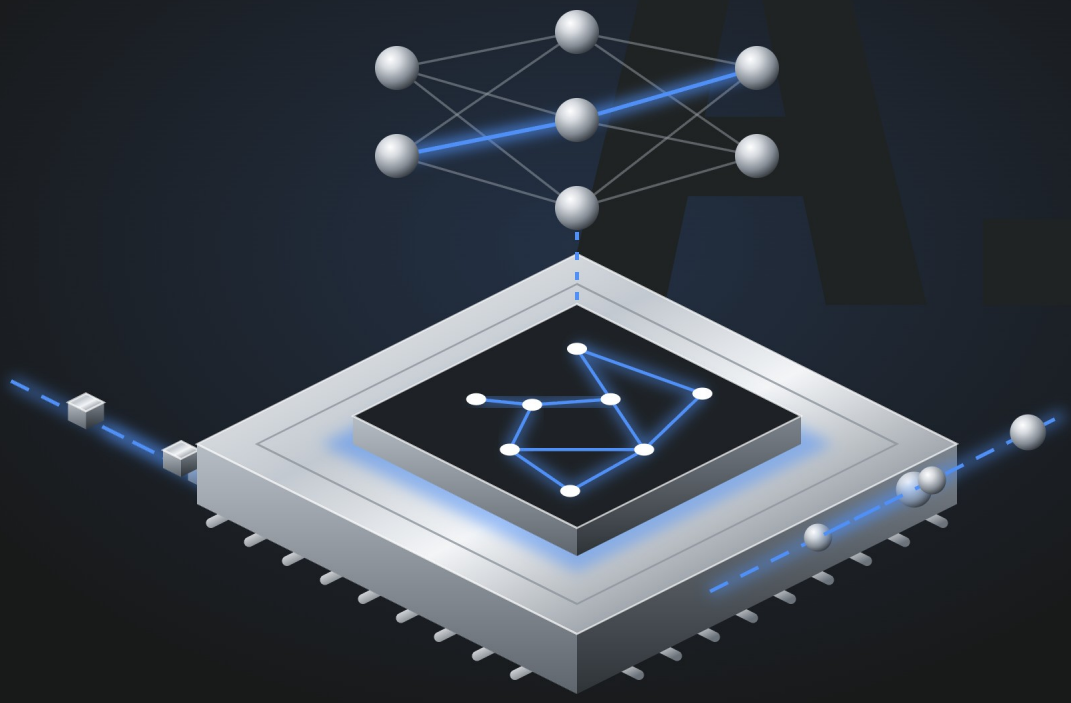
r_d

2-й потік

Міні-гайд



Підготовка до старту курсу

ГОТУЄМОСЬ ДО СТАРТУ

AI ENGINEERING

НАВІЩО ЦЕЙ ГАЙД

Лектор радить перед стартом освіжити основи Python і FastAPI, базові поняття ML, взаємозв'язок ML та AI і базову математику. Тут — короткий маршрут: що повторити, що відкрити і як перевірити себе перед стартом курсу.

ЯКИЙ ТРЕК ОБРАТИ

МІНІМАЛЬНИЙ

Щодня пишете на Python і вже вивчали ML

Пройдіть чекпоінти всіх розділів. Матеріали відкривайте лише там, де чекпоінт не виходить самостійно

РОЗШИРЕНИЙ

Python знаєте, але давно не працювали з FastAPI, ML чи математикою

Опрацюйте матеріали з позначкою «почніть з цього» і виконайте всі чекпоінти

Якщо можете виконати чекпоінт розділу, не заглядаючи в матеріали, — розділ можна пропустити.

↗ **приклад посилання** Так виглядають посилання на ресурси — натисніть на плашку, щоб відкрити матеріал.

ЗМІСТ

01	Основи Python	с. 3
02	Основи FastAPI	с. 4
03	Базові поняття Machine Learning	с. 5
04	Взаємозв'язок ML та AI	с. 6
05	Базова математика для AI	с. 7
06	Claude Code — обов'язковий інструмент	с. 8
07	Середовище й курсовий проєкт	с. 9
✓	Міні-задача для самоперевірки	с. 10
✓	Чекліст готовності	с. 11

01

ОСНОВИ PYTHON

Python — робоча мова всіх кейсів курсу, тому важливий не синтаксис «з нуля», а навички написання продакшн-коду: типізація, структура проєкту, віртуальні середовища, async.



ЩО САМЕ ПОВТОРИТИ

- Колекції: list / dict / set, comprehensions, генератори, `enumerate`, `zip`
- Функції: `*args`, `**kwargs`, значення за замовчуванням, lambda
- Класи та `@dataclass`, базове наслідування
- Type hints (`list[str]`, `Optional`) — на них побудовані FastAPI і Pydantic
- Модулі й пакети; `venv` або `uv`, `requirements.txt`
- Обробка винятків, робота з файлами та JSON
- `async` / `await` — що це і навіщо (детально — на занятті 8)
- `numpy` (масиви) і `pandas` (DataFrame, фільтрація, groupby)

ЧЕКПОІНТ

- [ ] Можу написати функцію з type hints, яка читає JSON-файл і повертає список `dataclass`-об'єктів
- [ ] Можу створити віртуальне середовище й встановити залежності з `requirements.txt`
- [ ] Пояснюю своїми словами, чим `async def` відрізняється від `def`

МАТЕРІАЛИ

почніть з цього

Kaggle Learn: Python

Швидке повторення основ в інтерактивних ноутбуках; нічого не потрібно встановлювати

↗ kaggle.com

Офіційний туторіал Python

Довідник: розділи 4–9 — функції, структури даних, модулі, винятки, класи

↗ docs.python.org

Kaggle Learn: Pandas

Робота з табличними даними — знадобиться для preprocessing та ML

↗ kaggle.com

02

ОСНОВИ FASTAPI

У другому блоці весь RAG-кейс обгортається в API на FastAPI. До старту достатньо вміти запустити простий сервіс і розуміти, як він приймає та повертає дані.



ЩО САМЕ ПОВТОРИТИ

- Принципи REST API: GET / POST / PUT / DELETE, статус-коди, JSON
- Перший застосунок: `FastAPI()`, декоратори маршрутів, `fastapi dev`
- Path- та query-параметри
- Тіло запиту через Pydantic-моделі (`BaseModel`) і валідація
- Автоматична документація `/docs` (Swagger UI)
- `async def` в ендпоінтах: коли він доречний

МІНІМАЛЬНИЙ СЕРВІС

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    text: str

@app.post("/predict")
def predict(item: Item):
    return {"length":
            len(item.text)}
```

Встановлення: `pip install "fastapi[standard]"`

Запуск: `fastapi dev main.py`

З uv: `uv run fastapi dev main.py`

ЧЕКПОІНТ

- [ ] Можу запустити локально сервіс з ендпоінтом `POST /predict`, який приймає `{"text": "..."}` і повертає кількість символів
- [ ] Можу відкрити `/docs` і протестувати ендпоінт звідти
- [ ] Розумію, що станеться, якщо надіслати `{"text": 123}` або порожнє тіло, і чому

МАТЕРІАЛИ

почніть з цього

FastAPI Tutorial – User Guide

Від First Steps до Request Body та Response Model

↗ fastapi.tiangolo.com

Concurrency and async / await

Пояснення async простими словами — на прикладі з бургерами

↗ fastapi.tiangolo.com

Pydantic: документація

Моделі даних і валідація (розділ Models)

↗ docs.pydantic.dev

03

БАЗОВІ ПОНЯТТЯ ML

На занятті 3 ви будуєте scikit-learn pipelines і MLflow-трекінг, на занятті 4 — пакуєте нейромодель. Щоб встигати за практикою, базові терміни ML мають бути добре знайомі.



ЩО САМЕ ПОВТОРИТИ

- Supervised vs unsupervised learning; регресія vs класифікація
- Ознаки (features), цільова змінна (target), параметри vs гіперпараметри
- Train / validation / test split і що таке data leakage
- Overfitting і underfitting, регуляризація на інтуїтивному рівні
- Функція втрат (loss) і градієнтний спуск
- Метрики: accuracy, precision, recall, F1, confusion matrix; MAE / MSE
- Нейромережа: шари, ваги, функція активації, epoch, batch

ЧЕКПОІНТ

- [ ] Пояснюю на прикладі, чим precision відрізняється від recall і коли важливіша кожна з цих метрик
- [ ] Знаю, чому не можна масштабувати дані (scaler) до розбиття на train / test
- [ ] Можу навчити `LogisticRegression` на датасеті з sklearn і порахувати метрики на тестовій вибірці

МАТЕРІАЛИ

почніть з цього

Google ML Crash Course

Модулі ML Models і Datasets, generalization, overfitting: відео, інтерактивні візуалізації, вправи в Colab

↗ developers.google.com

Kaggle: Intro to Machine Learning

Перша модель на scikit-learn, validation, underfitting / overfitting — на практиці

↗ kaggle.com

scikit-learn: Getting Started

`fit` / `predict`, `Pipeline`, cross-validation — саме те, що буде на занятті 3

↗ scikit-learn.org

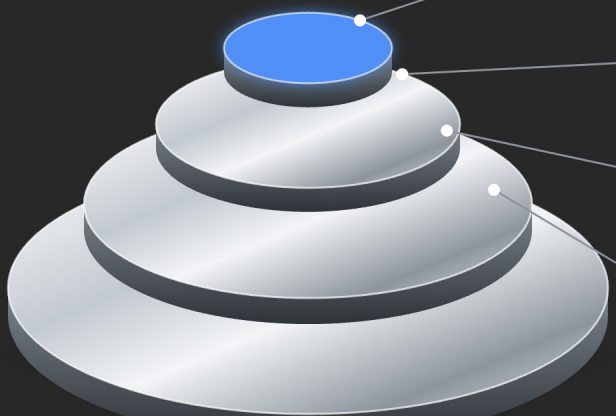
Давно не працювали з `numpy` / `pandas`?
У Google MLCC є короткі Colab-туторіали з обох бібліотек.

↗ Prerequisites and prework

04

ВЗАЄМОЗВ'ЯЗОК ML ТА AI

Спрощено ці поняття можна уявити як вкладені одне в одне. На занятті 1 саме від цього відштовхуватимемося, коли розбиратимемо, чим AI Engineering відрізняється від ML / Data / Backend.



LLM
Трансформери, що передбачають наступний токен; API або self-hosted

DEEP LEARNING
Багатошарові нейромережі, що самі виділяють ознаки з сирих даних

MACHINE LEARNING
Модель навчається на даних, а не діє за правилами, написаними вручну

AI
Будь-яка «інтелектуальна» система, зокрема й на правилах, без навчання

МІСЦЕ AI ENGINEER У ЦІЙ СХЕМІ

Він рідко навчає моделі з нуля, натомість будує продукт навколо них: дані, RAG, агенти, API, деплой, моніторинг. Терміни з перших блоків:

токен · embedding · контекстне вікно · inference vs training · fine-tuning

ЧЕКПОІНТ

- [ ] Можу за 2 хвилини пояснити колезі, чим ML відрізняється від програмування на правилах
- [ ] Можу навести приклад AI-системи, яка не є ML, і ML-моделі, яка не є нейромережею
- [ ] Розумію, що таке токен і чому вартість LLM-запиту рахується в токенах

МАТЕРІАЛИ

почніть з цього

Karpathy — Intro to Large Language Models

Як влаштовані та як навчаються LLM і де межі їхніх можливостей

↗ youtube.com

3Blue1Brown — Neural Networks

Перші два відео та відео про трансформери

↗ 3blue1brown.com

Google MLCC: Large Language Models

Модуль Advanced ML Models, розділ про LLM

↗ developers.google.com

05

БАЗОВА МАТЕМАТИКА ДЛЯ AI



Доводити теореми не потрібно, але важливо інтуїтивно розуміти, що відбувається «під капотом». Найчастіше знадобляться вектори та градієнти.

РОЗДІЛ	ПОНЯТТЯ	ДЕ НА КУРСІ
Лінійна алгебра	Вектор, матриця, множення матриць, скалярний добуток, норма	Embeddings, шари нейромережі, batching
Метрики подібності	Косинусна подібність vs евклідова відстань	Similarity search, векторні БД
Математичний аналіз	Похідна, градієнт, ланцюгове правило — на рівні інтуїції	Градієнтний спуск, навчання нейромереж
Ймовірності й статистика	Розподіл, середнє, медіана, дисперсія, умовна ймовірність	Softmax, метрики, оцінювання, drift

ФОРМУЛА ДО ЗАНЯТТЯ 6

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \ \|B\|}$$

Косинусна подібність порівнює напрямки двох векторів. Для embeddings вище значення зазвичай означає більшу семантичну подібність текстів.

ЧЕКПОІНТ

- [ ] Можу вручну порахувати скалярний добуток [1, 2, 3] і [4, 5, 6]
- [ ] Пояснюю, чому для текстів частіше беруть косинусну подібність, а не евклідову відстань
- [ ] Розумію, що показує градієнт і чому модель рухається в протилежному напрямку

МАТЕРІАЛИ

почніть з цього

Essence of Linear Algebra

3Blue1Brown: вектори, матриці, скалярний добуток — глави 1–4 і 9

↗ 3blue1brown.com

Essence of Calculus

3Blue1Brown: похідна та ланцюгове правило — глави 1–4

↗ 3blue1brown.com

Mathematics for ML

Книга-довідник: відкривайте потрібну главу, коли щось незрозуміло

↗ mml-book.github.io

06 обов'язково

CLAUDE CODE



Агентний AI-асистент для програмування від Anthropic: читає файли проєкту, редагує код, запускає команди й працює з Git. На попередньому потоці він був незамінним помічником студентів при створенні курсового проєкту.

ЩО ОСВОЇТИ ДО СТАРТУ

1. Встановіть Claude Code і увійдіть в акаунт: термінал, VS Code, PyCharm, десктоп або браузер
2. Запустіть `claude` у папці проєкту й попросіть пояснити структуру коду
3. Дайте невелике завдання, наприклад «додай ендпоінт /health», і перегляньте зміни перед підтвердженням
4. Зробіть commit через Claude Code

ВАЖЛИВО ПРО ДОСТУП

Потрібен платний план Claude (Pro, Max, Team або відповідний Enterprise-доступ) чи доступ через Claude Console з оплачуваними API-кредитами. Безкоштовного плану недостатньо.

КЛЮЧОВА НАВИЧКА

Чітко формулювати задачу й перевіряти результат. Код асистента треба розуміти, а не просто приймати — тому лектор пояснює не лише «що», а й «чому».

МАТЕРІАЛИ

почніть з цього

Claude Code Quickstart

Встановлення, вхід, перше завдання, робота з Git

↗ code.claude.com

Claude Code Overview

Способи запуску й безкоштовні курси Claude Academy

↗ code.claude.com

ЧЕКПОІНТ

- [ ] `claude --version` виводить номер версії
- [ ] Міні-задачу зі с. 10 виконано з Claude Code, і я розумію кожен рядок коду
- [ ] Знаю, як переглянути й відхилити запропоновану зміну

07

СЕРЕДОВИЩЕ Й КУРСОВИЙ ПРОЄКТ



Уже на першому занятті ви оберете наскрізний проєкт і створите перший чорновий pipeline, тому краще прийти з налаштованим робочим середовищем.

СЕРЕДОВИЩЕ

PYTHON + IDE

Python 3.11 або новіший, VS Code або PyCharm

GIT + GITHUB

Акаунт створено, вмієте створити репозиторій і зробити push — так здаються всі кейси

DOCKER DESKTOP

`docker run hello-world` виконується без помилок

Docker не потрібно знати заздалегідь, але Docker Desktop має бути встановлений і запускатися без помилок. Роботу з Dockerfile розберемо на курсі.

ЯКЩО ДАВНО НЕ ПРАЦЮВАЛИ З GIT АБО DOCKER

українською

Pro Git українською

Глави 1–3: commit, branch, merge, віддалений репозиторій

↗ git-scm.com

Docker: Get started

Образ і контейнер, Dockerfile, запуск контейнера

↗ docs.docker.com

ПОДУМАЙТЕ ЗАЗДАЛЕГІДЬ ПРО КУРСОВИЙ ПРОЄКТ

Це повноцінна AI-система, наближена до реального продукту. На занятті 1 потрібно обрати один із трьох типів:

RAG-СИСТЕМА

Відповідає на запитання з вашої бази документів

ML API

Модель, обгорнута в production-сервіс

LLM-АСИСТЕНТ

Агент, що працює з інструментами

Найкраще підходить тема з вашої роботи або сфери, де є реальні документи чи дані: внутрішня база знань, технічна документація, тікети підтримки.

за бажанням

МІНІ-ЗАДАЧА ДЛЯ САМОПЕРЕВІРКИ



Якщо ви впораєтеся з нею, не заглядаючи в туторіали, — ви готові до старту. У мініатюрі вона відтворює те, що на курсі ми робитимемо в продакшн-масштабі.

1. Завантажте датасет `load_iris` або `load_breast_cancer` з `sklearn.datasets`, розбийте на train / test.

2. Зберіть `Pipeline` зі `StandardScaler` і `LogisticRegression`, навчіть і виведіть precision, recall та F1 на тестовій вибірці через `classification_report`.

3. Збережіть модель у файл через `joblib`.

4. Запустіть FastAPI-сервіс з ендпоінтом `POST /predict`: він приймає ознаки об'єкта як Pydantic-модель і повертає передбачений клас.

5. Додайте ендпоінт `POST /similarity`: приймає два вектори однакової довжини й повертає їхню косинусну подібність, обчислену за допомогою `numpy`.

6. Протестуйте обидва ендпоінти через `/docs` і завантажте проєкт у GitHub-репозиторій.

ВИКОНУЙТЕ З CLAUDE CODE

Це водночас і перевірка роботи з інструментом: просіть асистента пояснювати кожен крок.

ЩОСЬ НЕ ВИЙШЛО?

Це нормально. Запишіть, на якому кроці зупинилися, і поставте запитання в каналі курсу або на першому занятті.

ВИ ГОТОВІ ДО СТАРТУ, ЯКЩО…

- [x] Python, IDE, Git і Docker встановлено; вмієте створити репозиторій на GitHub і зробити push
- [x] Claude Code встановлено, доступ до акаунта є
- [x] Запускаєте FastAPI-сервіс і тестуєте його через /docs
- [x] Пояснюєте precision / recall, overfitting і data leakage
- [x] Розумієте, що таке токен, embedding і косинусна подібність
- [x] Маєте ідею для курсового проєкту: RAG-система, ML API чи LLM-асистент
- [+] **Бонус:** виконали міні-задачу разом із Claude Code

УСПІШНОГО НАВЧАННЯ
ТА ДО ЗУСТРІЧІ НА КУРСІ!

r_d